

Задача: Распределённая система обмена сообщениями с персистентным хранением и оптимизацией работы с базой данных

Цель работы

Разработать высоконагруженную отказоустойчивую систему адресного обмена сообщениями с использованием WebSocket (`ws`), многопоточностью (`cluster` в Node.js) и асинхронной работой с базой данных через выделенный поток. Система должна гарантировать доставку сообщений при временном отсутствии клиента и попарное равенство отправленных и полученных сообщений.

Условие задачи

1. WebSocket-сервер (многопоточный с хранилищем)

- Количество воркеров сервера = количеству логических ядер CPU.
- Каждый воркер обрабатывает WebSocket-соединения и ведёт локальный кэш активных клиентов.
- При поступлении сообщения для неподключённого клиента сообщение сохраняется в базу данных.
- При (пере)подключении клиента сервер запрашивает через выделенный БД-воркер все не доставленные сообщения и отправляет их клиенту.

2. Выделенный поток для работы с базой данных (БД-воркер)

- Создаётся один отдельный воркер (например, `db-worker`), который не занимается обработкой WebSocket-соединений.
- Задачи БД-воркера:
 - Сохранение сообщений для офлайн-клиентов.
 - Извлечение накопленных сообщений при подключении клиента.
 - Атомарное удаление доставленных сообщений.
 - Ведение попарной статистики (опционально, для верификации).
- Серверные воркеры не обращаются к БД напрямую — только через IPC-запросы к БД-воркеру.
- БД-воркер должен использовать пул соединений и пакетную запись/чтение (batch operations).

3. Клиентские воркеры (10 штук)

- Имена генерируются главным потоком (`client-0` ... `client-9` or 928 00 000 00).
- Каждый клиент получает от master список имён других клиентов (9 имён).
- Клиенты:
 - Подключаются к серверу и регистрируются.
 - Получают от сервера все сохранённые для них сообщения (если есть).
 - Отправляют 100 000 сообщений случайным другим клиентам.
 - Ведут матрицу попарных счётчиков отправленных/полученных.
 - При разрыве соединения пытаются переподключиться с экспоненциальной задержкой.

4. Обработка отсутствующего клиента (offline-очередь)

Дополнительные задания для повышенной оценки

Реализовать горизонтальное масштабирование (несколько экземпляров приложения, общая БД)

Распределённая отказоустойчивая система обмена сообщениями с БД, случайными отключениями и автономными клиентскими процессами

Цель работы

Разработать высоконагруженную отказоустойчивую систему адресного обмена сообщениями с WebSocket (`ws`), многопоточностью (`cluster`), хранением в БД, демонстрацией сохранения сообщений при случайных отключениях и возможностью запуска дополнительных клиентских процессов вручную для прямого обмена данными.

1. WebSocket-сервер (многопоточный)

- Количество воркеров = число ядер CPU.
- Каждый воркер:
 - Принимает WebSocket-соединения.
 - Ведёт локальный кэш активных клиентов.
 - При недоступности получателя отправляет сообщение в БД-воркер.
- При подключении клиента запрашивает у БД-воркера все накопленные сообщения и отправляет их.

2. Выделенный БД-воркер

- Один воркер (отдельный процесс).
- Пул соединений .
- Пакетная запись *.
- Индексы по полю `to\_name`.
- Поддерживает операции:
 - `store\_offline` — сохранить сообщение.
 - `fetch\_offline` — получить все сообщения для клиента.
 - `delete\_offline` — удалить доставленные.
 - `get\_stats` — получить попарную статистику (опционально).

3. Клиентские воркеры (10 штук) — РАНДОМНОЕ ПЕРЕПОДКЛЮЧЕНИЕ

Каждый клиент периодически случайным образом отключается и переподключается, чтобы продемонстрировать, что сообщения сохраняются в БД и доставляются после восстановления.

Требования:

- Клиент подключается → регистрируется → получает накопленные сообщения.
- Каждые от 5 до 30 секунд (случайный интервал) клиент разрывает соединение (вызывает `ws.close()`).
- Через случайную задержку (от 2 до 10 секунд) переподключается заново.
- Клиент не теряет счётчики отправленных/полученных сообщений между переподключениями.
- После переподключения клиент получает от сервера все сообщения, которые были отправлены ему в офлайн-режиме.

Часть 2. Автономные клиентские процессы (дополнительное требование)

Показать, что система поддерживает не только воркеры, созданные главным процессом, но и внешние независимые процессы, которые могут:

- запускаться вручную (из командной строки);
- подключаться к тому же WebSocket-серверу;
- регистрироваться с уникальными именами;
- обмениваться сообщениями с основными клиентами и между собой.

Важные замечания

Внешние клиенты должны уметь подключаться к уже работающему серверу и не зависеть от главного процесса.

Сохранение состояния внешнего клиента в файл — упрощает демонстрацию (при перезапуске клиент помнит, сколько он уже отправил/получил).

Для проверки преподаватель может сам отключать любого клиента (послать SIGINT или закрыть терминал) — система должна корректно восстановить доставку.

Преподаватель обладает дискреционными полномочиями по оценке качества ответа, руководствуясь собственным профессиональным убеждением и установленными критериями, и вправе выставить оценку в соответствии с внутренним убеждением, основанным на содержательной полноте ответа.